9. Future Plans

The initial version of the E-Commerce Multi-Vendor Platform functions as a proof-of-concept. The following sections outline planned upgrades to evolve the application into a production-grade system.

9.1 Payment Gateway Integration (Razorpay)

Current Limitation: The system simulates transactions using Cash on Delivery (COD) without real money movement.

Planned Enhancement: Integrate the Razorpay SDK to handle digital payments (Cards, UPI, Wallets, Netbanking) securely.

```
import razorpay
from rest_framework.response import Response

client = razorpay.Client(auth=(RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET))

def create_payment_order(total_amount, order_id):
    order = client.order.create({
        "amount": int(total_amount * 100),  # amount in paise
        "currency": "INR",
        "receipt": f"order_\${order_id}",
    })
    return Response({"razorpay_order_id": order["id"]})
```

9.2 Mobile Application (React Native)

Planned Enhancement: Develop a cross-platform mobile application using React Native. Since the Django REST backend is stateless and communicates via JSON, the existing APIs support mobile clients without modifications.

- **Push Notifications:** Notify customers of order status updates using Firebase Cloud Messaging (FCM).
- **Device Integration:** Allow vendors to capture and upload product images using the device camera.
- **Biometric Security:** Implement FaceID/Fingerprint logins using native device APIs.

9.3 Production Cloud Deployment (AWS)

Planned Stack: Transition from local deployment to a scalable cloud infrastructure using Amazon Web Services:

Service Component	AWS Hosting Choice	Infrastructure Purpose
Django API Backend	AWS EC2 + Gunicorn + Nginx	Serves stateless API requests.
Relational Database	AWS RDS (PostgreSQL)	Production database with auto-backups.
Static/Media Uploads	AWS S3 + CloudFront CDN	Stores product media assets with low latency.
React SPA Client	Vercel / Netlify	Serves static React build assets globally.
DNS & SSL	AWS Route 53 + Certificate Mgr	Handles custom domain registration and HTTPS security.

9.4 Product Image Upload (AWS S3)

Current Limitation: The frontend displays static fallback emojis for product photos.

Planned Upgrade: Integrate `django-storages` and Pillow to process, compress, and upload product images directly to S3.

```
from django.db import models

class Product(models.Model):
    # ... existing fields ...
    image = models.ImageField(
        upload_to='products/',
        null=True,
        blank=True,
        help_text="Primary product image"
    )
```

9.5 Typo-Tolerant Search (Elasticsearch)

Planned Enhancement: Replace Django's SQL `icontains` search with Elasticsearch using `django-elasticsearch-dsl`.

- **Fuzzy Matching:** Searches tolerate common typos (e.g., "iphon" still matches "iPhone").
- **Relevance Sorting:** Orders search results by search score.
- **Autocomplete:** Displays suggestions dynamically as the user types.

9.6 Summary of Future Upgrades

ID	Upgrade Plan	Technology Stack	Priority
9.1	Razorpay Gateway Integration	Razorpay Python SDK + React Checkout	HIGH
9.2	React Native Mobile App	React Native + Expo Framework	HIGH
9.3	AWS Production Cloud	AWS (EC2, RDS, S3, CloudFront)	HIGH
9.4	S3 Product Image Uploads	AWS S3 + Pillow + Django Storages	MEDIUM
9.5	Typo-Tolerant Search	Elasticsearch + Django Search DSL	MEDIUM
9.6	Customer Reviews & Ratings	Django Review Model + React Stars UI	MEDIUM
9.7	Interactive Admin Charts	Recharts (React D3 charting library)	LOW